

**A Mini Project Report on**

**“Deepfake Detector: A CNN-BiLSTM Attention-Based  
Web Application for Image and Video Forgery Detection”**

**Submitted in partial fulfillment of the requirement for Degree in Bachelor  
of Technology (Information Technology)**

**By**

Abhishek Tajane (5023158)

**Guided by:**

Dr. Mukta Nivelkar



**Department of Information Technology**

**Fr. C. Rodrigues Institute of Technology, Vashi.**

(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

Academic Year: 2025-26

# **CERTIFICATE**

This is to certify that the Mini Project entitled

**“Deepfake Detector: A CNN-BiLSTM Attention-Based  
Web Application for Image and Video Forgery Detection”**

**Submitted By**

Abhishek Tajane (5023158)

In partial fulfillment of degree of **B.Tech. in Information Technology** for term work of the  
Semester VI Mini Project – 2 B is approved.

**Internal Examiner**

**Internal Guide**

**Head of the Department**

**Principal**

**Date: -**

**College Seal**

## **Declaration**

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Abhishek Tajane (5023158)

Date:

## **Abstract**

In today's digital era, misinformation and manipulated media have become major challenges across social platforms. With the rise of artificial intelligence, deepfake images are increasingly being used to spread false information, damage reputations, and mislead audiences. To address this issue, we are developing a simple and effective solution — a Deepfake Detector that helps people verify the authenticity of digital images quickly and accurately.

The main goal of Deepfake Detector is to create a web application that can analyze uploaded images using artificial intelligence. It employs a Convolutional Neural Network (CNN) model trained on real and fake image datasets to detect manipulation or forgery. The system generates a confidence score indicating whether an image is likely to be genuine or deepfake, empowering users to verify visual content before trusting or sharing it online.

The application is designed to be user-friendly and accessible from any device. With real-time image analysis and clear results, it simplifies the process of verifying authenticity for individuals, journalists, and organizations alike. Overall, Deepfake Detector aims to promote digital safety and awareness by helping users distinguish between real and AI-generated media.

## INDEX

| Sr. No. | Topic                                                                                                                                                                                           | Page No.                   |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 1       | Introduction <ul style="list-style-type: none"><li>- Background</li><li>- Motivation</li><li>- Problem Definition</li><li>- Scope</li><li>- Issues / Limitation</li></ul>                       | 1<br>2<br>2<br>5<br>5<br>5 |
| 2       | Literature Survey and Analysis <ul style="list-style-type: none"><li>- Referred paper</li><li>- Literature Survey Comparison</li><li>- Existing System</li><li>- Requirement Analysis</li></ul> | 7<br>8<br>10<br>14<br>14   |
| 3       | Architectural Design <ul style="list-style-type: none"><li>- Architectural Diagram / block diagram</li><li>- Hardware &amp; Software Requirement</li><li>- Flow chart</li></ul>                 | 16<br>17<br>18<br>19       |
| 4       | User Interface                                                                                                                                                                                  | 20                         |
| 5       | Results                                                                                                                                                                                         | 24                         |
| 6       | Conclusion                                                                                                                                                                                      | 26                         |
|         | References                                                                                                                                                                                      | 28                         |
|         | Acknowledgements                                                                                                                                                                                | 29                         |
|         | Appendix                                                                                                                                                                                        | 30                         |

## LIST OF FIGURES

| <b>Fig. No.</b> | <b>Name of the Figure</b>                      | <b>Page No.</b> |
|-----------------|------------------------------------------------|-----------------|
| 1.1             | Visual representation of Deepfake Manipulation | 3               |
| 1.2             | Growth of Deepfake and Identity Fraud cases    | 4               |
| 2.2             | EDA: Pixel Statistics (Real vs. Fake)          | 15              |
| 3.1             | Architecture Diagram                           | 17              |
| 3.3             | Flow Chart                                     | 18              |
| 4.1             | TruthLens Landing Page (Home)                  | 20              |
| 4.2             | Deepfake Detector Upload Interface             | 20              |
| 4.3             | Deepfake Detected (Image)                      | 21              |
| 4.4             | Authentic Image Verified                       | 21              |
| 4.5             | Deepfake Detected (Video)                      | 22              |
| 4.6             | Authentic Video Verified                       | 22              |
| 5.1             | Training Curves of Baseline CNN Model          | 24              |
| 5.2             | Confusion Matrix and ROC Curve                 | 25              |
| 5.3             | Classification Report                          | 25              |

**LIST OF TABLES**

| <b>Table No.</b> | <b>Name of the Table</b> | <b>Page No.</b> |
|------------------|--------------------------|-----------------|
| 2.1              | Literature Survey        | 10              |

# **CHAPTER 1**

## **INTRODUCTION**



# INTRODUCTION

## 1.1 Background

With the rapid advancement of artificial intelligence, *deepfake technology* has made it possible to generate highly realistic but fake videos and images. These manipulations can easily deceive the human eye and are often misused for spreading misinformation, identity theft, and defamation.

The Deepfake Detection System is an AI-based solution designed to detect and analyze such manipulated media using deep learning and computer vision. The system leverages Convolutional Neural Networks (CNNs) trained on datasets such as FaceForensics++ to identify subtle inconsistencies and rendering artifacts that differentiate fake content from real ones.

It also includes a user-friendly web interface built with React.js and a Python (Flask/FastAPI) backend, allowing users to upload media for verification and receive instant results.

## 1.2 Motivation

Deepfakes pose a major threat to digital trust, enabling fake news, cyber fraud, and identity manipulation. With social media platforms becoming the main source of information, verifying content authenticity has become critical. As illustrated below in *Figure 1.1*, the visual difference between real and manipulated videos can be nearly indistinguishable, emphasizing the urgent need for automated detection systems.

Our motivation is to restore trust in digital content by developing an intelligent system capable of detecting deepfakes quickly and accurately. The project aims to:

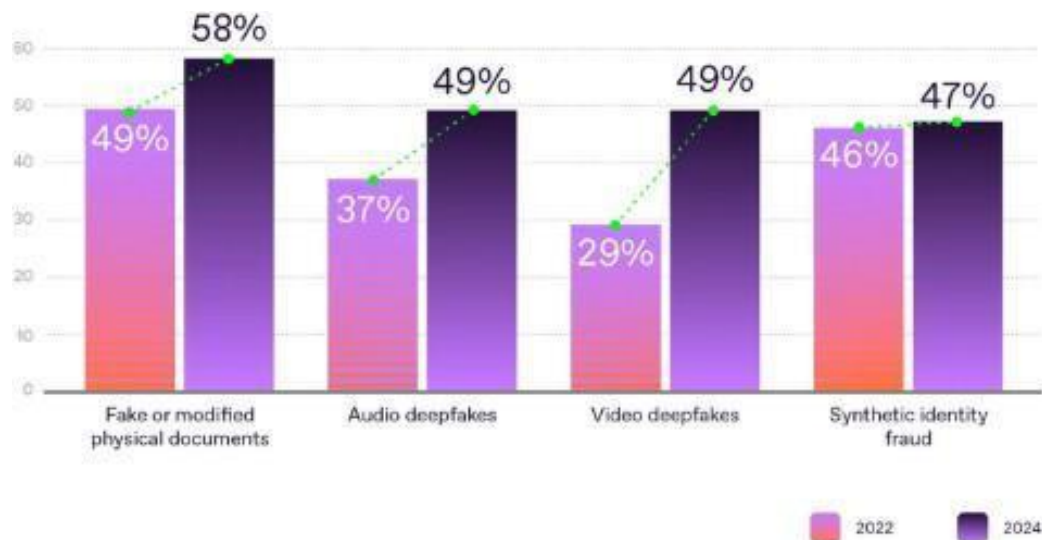
1. **Ensure Safety:** Protect individuals and organizations from misinformation and reputational harm.
2. **Promote Digital Trust:** Empower users to verify online content before sharing or believing it.
3. **Support Sustainable Online Practices:** Encourage responsible use of AI and media through detection tools.
4. By combining deep learning techniques with a real-time detection platform, this project contributes to a safer digital ecosystem and aligns with **UN SDG 16 (Peace, Justice, and Strong Institutions)** and **SDG 12 (Responsible Consumption and Production)**.



**Figure 1.1 – Visual Representation of Deepfake Manipulation**

### **1.2.1 Key Motivations**

1. **Digital Safety:** Deepfakes can be used for misinformation, blackmail, and identity theft. Detecting them early helps protect individuals and organizations from serious harm.
2. **Restoring Trust:** Online platforms are flooded with manipulated media. This project promotes transparency and helps rebuild trust in digital content.
3. **Technological Awareness:** Educating people about AI-generated content and giving them tools to verify authenticity increases awareness of media manipulation.
4. **Accessibility:** By providing a simple web-based interface, anyone can upload and verify videos or images — making AI detection tools available to all.



**Figure 1.2 Growth of Deepfake and Identity Fraud Cases from 2022 to 2024**

As shown in the chart *Figure 1.2*, the prevalence of deepfakes and related frauds has sharply increased between 2022 and 2024 — with audio and video deepfakes rising from 37% and 29% to 49%, respectively. This alarming trend highlights the urgent need for reliable detection systems. Digital safety is the foremost motivation, as deepfakes are increasingly used for misinformation, blackmail, and identity theft. Detecting them early helps protect individuals and organizations from significant harm. Equally important is restoring trust in online media; with manipulated content spreading rapidly, this project promotes transparency and ensures that digital information remains credible.

Another major goal is technological awareness — empowering users to understand and identify AI-generated media. The system’s accessible web-based interface allows anyone, not just experts, to upload and verify images or videos easily. Finally, this project reflects strong social responsibility, supporting global Sustainable Development Goals such as SDG 16 (Peace, Justice, and Strong Institutions) by combating misinformation, and SDG 12 (Responsible Consumption and Production) by encouraging ethical use of technology.

## 1.3 Problem Definition

With the rise of artificial intelligence, *deepfake* technology has made it possible to create highly realistic fake videos and images that are difficult for the human eye to distinguish from real ones. These deepfakes can be misused for spreading misinformation, defamation, financial scams, identity theft, and other cybercrimes. Detecting such manipulations manually is nearly impossible, creating a serious threat to digital trust and online authenticity.

There is a need for an AI-based solution that can automatically identify deepfake content and provide users with an accurate and reliable verification system.

## 1.4 Scope

The main objective of this project is to build an AI-powered application that detects whether an image or video is real or artificially manipulated. The specific objectives are as follows:

1. **Deepfake** **Detection** **System:**  
To develop an intelligent system capable of identifying whether an image or video is authentic or manipulated using advanced AI techniques.
2. **User** **Input** **Support:**  
To allow users to upload or capture images and videos through the application for verification.
3. **AI-Based** **Analysis:**  
To utilize deep learning models such as Convolutional Neural Networks (CNN) and transfer learning techniques to detect facial and visual manipulations.
4. **Real-Time** **Prediction:**  
To provide instant predictions along with a confidence score (%) indicating the likelihood of content being a deepfake.
5. **Multi-Format** **Support:**  
To support both image-level and video-level detection for enhanced flexibility and usability.
6. **Misinformation** **Reduction:**  
To minimize the risk of misinformation, fraud, and misuse by enabling users to verify digital content authenticity.
7. **Digital** **Safety** **Enhancement:**  
To improve public trust and digital safety by offering a simple, reliable, and accessible verification tool.
8. **Backend** **Processing:**  
To implement cloud-based backend APIs (such as Flask or FastAPI) for efficient handling of computationally intensive AI tasks.
9. **Data** **Logging** **(Optional):**  
To store user activity logs and analysis history for future reference and continuous model improvement.
10. **Scalability:**  
To ensure the system can scale effectively to support more users, larger datasets, and evolving deepfake detection techniques over time.

## 1.5 Issues / Limitations

Despite the effectiveness of the proposed deepfake detection system, certain limitations exist:

1. **Sensitivity to Media Quality:**  
Detection accuracy may decrease when processing low-quality, highly compressed, or blurry images and videos.
2. **Advanced Manipulation Techniques:**  
Highly sophisticated or previously unseen deepfake techniques may not be accurately recognized by the model.
3. **Dataset Dependency:**  
The system relies heavily on large labeled datasets (e.g., FaceForensics++). Limited diversity in training data may affect the model's ability to generalize.
4. **Need for Continuous Updates:**  
The model requires periodic retraining to adapt to newly emerging deepfake generation methods.
5. **Computational Requirements:**  
Real-time video detection can be computationally expensive, especially on low-end or resource-constrained devices.
6. **Probabilistic Predictions:**  
The model provides confidence scores rather than absolute certainty, which may lead to ambiguous interpretations in borderline cases.
7. **Scope Limitation:**  
The current system focuses primarily on face-based manipulations and may not effectively detect non-face or audio-based deepfakes in its initial version.

# **CHAPTER 2**

## **LITERATURE SURVEY**

## LITERATURE SURVEY

- 1. FaceForensics++ (2019)** — Rössler et al. Built a large-scale benchmark dataset with manipulated facial images and tested multiple CNN architectures on it. Became the standard reference dataset for deepfake detection research.
- 2. MesoNet (2018)** — Afchar et al. Designed a compact, lightweight CNN (Meso-4 / MesoInception-4) that detects facial forgeries in real time. Trades off accuracy for speed and model size.
- 3. FakeCatcher (2019/2020)** — Ciftci et al. Detects deepfakes using biological signals — specifically blood flow patterns (rPPG) extracted from skin regions — rather than visual artifacts. Hard for generators to replicate.
- 4. Face X-Ray (2020)** — Li et al. Identifies blending boundary artifacts left during face swapping. Generalizes across unseen manipulation types but fails on fully synthesized (non-blended) fakes.
- 5. Frequency Analysis for Deepfake Recognition (2020)** — Frank et al. Exploits unique spectral artifacts in GAN-generated images. Works across multiple generators without retraining but may fail against diffusion-based fakes.
- 6. Thinking in Frequency (ECCV 2020)** — Qian et al. Adds frequency-aware layers to CNNs so the network jointly learns spatial and frequency cues. Improves generalization but increases compute cost.
- 7. Exploiting Temporal Information (IEEE ICIP)** — Guera & Delp Combines CNN (spatial features per frame) with LSTM (temporal inconsistency across frames). Simple and clean pipeline but uses older CNN backbones.
- 8. Deepfake Video Detection Using RNNs (IEEE AVSS)** — Sabir et al. CNN extracts frame-level features, LSTM classifies at video level. Effective for video-level detection but computationally heavy.
- 9. CNN + LSTM for Video Forgery Detection** — Zhou et al. Hybrid CNN-LSTM framework focusing on spatio-temporal forgery detection. Improves accuracy but increases memory usage and pipeline complexity.
- 10. Deepfake Detection Using Spatiotemporal Features (IEEE Access)** — Li et al. Jointly trains CNN and LSTM to capture both spatial and temporal artifacts. Robust detection but struggles to generalize across datasets.
- 11. Convolutional Vision Transformer / CViT (2021)** — Wodajo & Atnafu Combines CNN for local feature extraction with Vision Transformer for global attention. Achieves 91.5% accuracy and AUC of 0.91 on DFDC but limited cross-dataset generalization.
- 12. Deepfake Detection Using CNN-Based Architecture (2025)** — Kapoor et al. Presents a complete end-to-end CNN pipeline with detailed preprocessing, architecture design, and training strategy. Practical and systematic but needs continuous updates as generation techniques evolve.

**13. A Survey: Deepfake and Current Technologies (2024)** — Banerjee et al. Surveys forensics, ML, multimodal, and blockchain-based detection approaches. Covers both technical and societal dimensions but real-time detection remains an open problem.

**14. DeepFakes and Beyond (2020)** — Tolosana et al. Classifies face manipulations into 4 types (full synthesis, identity swap, attribute manipulation, expression swap) and reviews all major datasets and benchmarks. Strong foundational reference.

**15. Deepfake Generation and Detection: A Benchmark and Survey (2024)** — Pei et al. Covers the full arc from GAN to diffusion-based generation and benchmarks detection methods across spatial, temporal, frequency, and hybrid domains. Most comprehensive and up-to-date survey.



### **Table 2.1 Literature Survey**

| Sr. No. | Paper / Year                                                                             | Authors                                                                                             | Description                                                                                                                 | Advantages                                                                                                                                  | Limitations                                                                                                                      |
|---------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 1       | FaceForensics++: Learning to Detect Manipulated Facial Images (2019)                     | Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, Matthias Nießner | Introduced a large-scale benchmark dataset for detecting manipulated facial images and evaluated various CNN architectures. | Standardized benchmark dataset; enables reproducible comparison across models; supports multiple manipulation types and compression levels. | Focuses mainly on early manipulation methods; detectors trained on it often overfit and fail to generalize to unseen generators. |
| 2       | MesoNet: A Compact Facial Video Forgery Detection Network (2018)                         | Darius Afchar, Vincent Nozick, Junichi Yamagishi, Isao Echizen                                      | Proposed a lightweight CNN (Meso-4, MesoInception-4) for detecting facial forgery in images and videos.                     | Small model, fast inference (real-time use); easy to interpret activations and visualize results.                                           | Lower accuracy than deeper models (Xception, ResNet); sensitive to compression and post-processing.                              |
| 3       | FakeCatcher: Detection of Synthetic Portrait Videos using Biological Signals (2019/2020) | Umur Aybars Ciftci, Ilke Demir, Lijun Yin                                                           | Uses biological signals (rPPG — blood flow patterns) from skin regions to detect fake videos or images.                     | Uses physiological cues that are hard for generative models to mimic; robust to some post-processing.                                       | Mainly applicable to videos, not single images; sensitive to lighting and motion artifacts.                                      |
| 4       | Face X-Ray for More General Face Forgery Detection (2020)                                | Lingzhi Li, Jianmin Bao, Ting Zhang, Hao Yang, Dong Chen, Fang Wen, Baining                         | Introduces a 'face X-ray' representation to identify image blending boundaries common in                                    | Generalizes across many manipulation types; localizes manipulated regions                                                                   | Ineffective for full-synthesis (non-blended) fakes; post-processing can hide boundary cues.                                      |

|   |                                                                                                |                                                                            |                                                                                                                    |                                                                                                        |                                                                                                                             |
|---|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
|   |                                                                                                | Guo                                                                        | manipulated faces.                                                                                                 | (interpretable).                                                                                       |                                                                                                                             |
| 5 | Leveraging Frequency Analysis for Deepfake Image Recognition (2020)                            | Jörg Frank, T. Eisenhofer, L. Schönherr, Asja Fischer, D. Kolossa, T. Holz | Shows that GAN-generated images have distinctive frequency artifacts; uses spectral-domain features for detection. | Model-agnostic — frequency features apply across many generators; good complement to CNN spatial cues. | Vulnerable if generator uses spectral regularization or improved upsampling; may fail on diffusion-based fakes.             |
| 6 | Thinking in Frequency: Face Forgery Detection via Frequency-aware Feature Learning (ECCV 2020) | Yuezun Qian, Xianfeng Zhao, Guojun Yin, Jiajun Wu, Siwei Lyu               | Improves CNNs by adding frequency-aware layers for detecting fake faces more robustly.                             | Learns robust frequency features jointly with spatial cues; better generalization across datasets.     | Requires high-quality input; performance degrades under compression; additional computation cost compared to standard CNNs. |
| 7 | Exploiting Temporal Information for Deepfake Detection (IEEE ICIP)                             | David Guera, Edward J. Delp                                                | Uses CNN to extract spatial frame features from video frames and LSTM to learn temporal inconsistencies.           | Clear CNN to LSTM pipeline; strong temporal modeling; easy to implement.                               | Uses older CNN architectures; comparatively lower accuracy.                                                                 |
| 8 | Deepfake Video Detection Using Recurrent                                                       | Sabir et al.                                                               | CNN-based frame feature extraction                                                                                 | Direct CNN + LSTM integration;                                                                         | High computational cost; longer training                                                                                    |

|    |                                                                                         |                                 |                                                                                                       |                                                                            |                                                       |
|----|-----------------------------------------------------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------|
|    | Neural Networks (IEEE AVSS)                                                             |                                 | followed by LSTM for video-level classification.                                                      | effective video-level detection.                                           | time.                                                 |
| 9  | Combining CNN and LSTM for Video-Based Face Forgery Detection (Elsevier Neurocomputing) | Zhou et al.                     | Hybrid CNN + LSTM framework for spatial and temporal forgery detection.                               | Strong spatio-temporal learning; improved accuracy.                        | More complex pipeline; higher memory usage.           |
| 10 | Deepfake Detection Using Spatiotemporal Features (IEEE Access)                          | Li et al.                       | Joint CNN and LSTM model to capture spatial and temporal artifacts in videos.                         | Explicit spatio-temporal modeling; robust detection.                       | Dataset-dependent performance; generalization issues. |
| 11 | Deepfake Video Detection Using Convolutional Vision Transformer (2021)                  | Deressa Wodajo & Solomon Atnafu | Proposes a CNN + Vision Transformer (CViT) hybrid model for deepfake video detection on DFDC dataset. | High accuracy (91.5%, AUC=0.91); leverages both local and global features. | Generalization to unseen datasets is still limited.   |

|    |                                                                                              |                                                                                             |                                                                                                                      |                                                                                                                         |                                                                                                                  |
|----|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| 12 | A Survey: Deepfake and Current Technologies for Solutions (2024)                             | Sayan Banerjee, Sumit Kumar Yadav, Ankit Dhara, Md Ajj                                      | Reviews detection methods including forensics, ML, multimodal approaches, and blockchain authentication.             | Broad coverage including technical and societal aspects.                                                                | Real-time detection remains unsolved.                                                                            |
| 13 | DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection (2020)                | Ruben Tolosana, Ruben Vera-Rodriguez, Julian Fierrez, Aythami Morales, Javier Ortega-Garcia | Categorizes 4 main manipulation types; reviews datasets, benchmarks, and detection methods.                          | Strong classification framework; benchmark results summarized.                                                          | Traditional methods are less robust against new manipulations.                                                   |
| 14 | Deepfake Generation and Detection: A Benchmark and Survey (2024)                             | Gan Pei, Jiangning Zhang, Menghan Hu et al.                                                 | Benchmarks detection methods; covers evolution from GAN to diffusion models; unifies deepfake task definitions.      | Unified definitions; cross-domain benchmarking; identifies future directions.                                           | Diffusion-based detection at early stage; challenges in real-world deployment.                                   |
| 15 | FakeFormer: Efficient Vulnerability-Driven Transformers for Generalizable Deepfake Detection | Viet-Tuan Le, Yibing Zhan, Xiangtai Li, Yunqian Li, Xiaochun Cao                            | Proposes a transformer-based detector that focuses on vulnerability-driven patches in video frames for generalizable | Strong cross-dataset generalization; efficient patch-based attention; outperforms CNN baselines on multiple benchmarks. | Higher memory requirements; transformer training demands large datasets; less interpretable than CNN approaches. |

|  |        |  |            |  |  |
|--|--------|--|------------|--|--|
|  | (2023) |  | detection. |  |  |
|--|--------|--|------------|--|--|

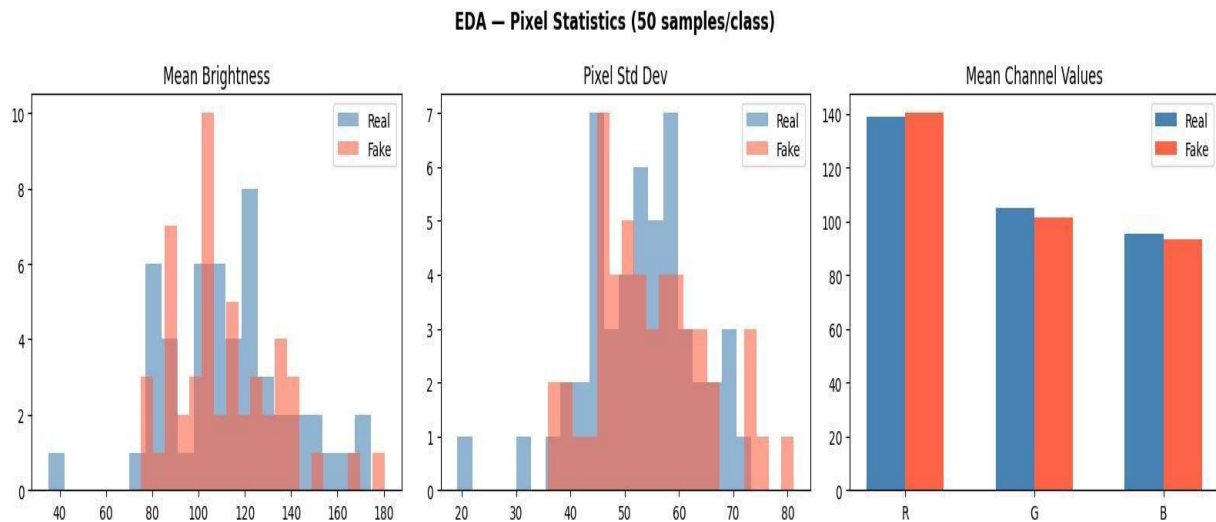
## 2.1 Existing Systems

Several existing platforms and tools offer deepfake detection capabilities for various use cases:

- i. **Sensity AI** – Enterprise-level media forensics platform for detecting synthetic media. Targets organizations needing advanced detection tools and forensic reporting.
- ii. **Reality Defender** – Provides API and browser plugin for real-time fake media detection. Integrates with workflows to deliver detection scores and content analysis.
- iii. **Deepware** – Publicly available video deepfake scanner focusing on face-swap techniques. Allows users to upload videos and receive detection reports.
- iv. **Truepic** – Ensures content authenticity and tracks media provenance. Aims to prevent manipulation and verify the integrity of digital content.
- v. **InVID/WeVerify** – Verification toolkit for journalists and fact-checkers. Offers multiple media analysis tools, including deepfake detection.

## 2.2 Requirement Analysis

- i. **Users will be provided with accessible deepfake detection through an intuitive web interface:**  
The system will allow users to upload images or videos without requiring technical expertise or API integration, guiding them through the upload process and presenting results with clear confidence scores.
- ii. **Users will be provided with transparent detection results that explain the basis for the system's conclusions:**  
The system will display confidence scores (e.g., "82% likely fake") along with explanations of factors contributing to the detection decision, promoting AI transparency.
- iii. **Users will be provided with rapid analysis capabilities for quick verification of digital content:**  
The system will process uploaded media efficiently, delivering detection results within seconds to minutes depending on file size, prioritizing real-time feedback.
- iv. **Users will be provided with a reporting mechanism to flag detected deepfakes and contribute to community awareness:**  
The platform will include functionality to report suspicious media, creating a feedback loop for improving detection models and supporting broader efforts to combat misinformation.
- v. **Users will be provided with detection capabilities that work across common deepfake manipulation techniques:**  
The system will be trained on diverse datasets (FaceForensics++, DFDC) using XceptionNet-based architecture to recognize various manipulation methods including face swaps, facial reenactment, and synthetic face generation.



**Figure 2.2–EDA : Pixel Statistics of Real vs. Fake Images(50 samples/class)**

As part of our preliminary data exploration, an Exploratory Data Analysis (EDA) was conducted on the dataset to understand pixel-level differences between real and deepfake images. As shown in Figure 2.2, the mean brightness, pixel standard deviation, and mean RGB channel values across both classes reveal subtle but consistent statistical differences — with fake images showing slightly more varied brightness distributions and marginally different channel intensities. These observations confirmed that spatial pixel features alone are insufficient for reliable deepfake detection, motivating the use of deeper feature extraction via CNNs.

## 2.3 Python Code

```
from __future__ import annotations
import argparse
import asyncio
import csv
import hashlib
import json
import logging
import os
import queue
import random
import sqlite3
import statistics
import threading
import time
import uuid
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Callable, Iterable, Iterator, Optional, Sequence
from contextlib import contextmanager
from functools import wraps
```

```
DEFAULT_BATCH_SIZE = 25
```



```

DEFAULT_RETRIES = 3
DEFAULT_TIMEOUT = 5.0
CACHE_TTL_SECONDS = 300
DB_SCHEMA_VERSION = 1

logger = logging.getLogger("pipeline")
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
)

@dataclass
class Settings:
    source: str = "input.jsonl"
    database: str = "pipeline.db"
    batch_size: int = DEFAULT_BATCH_SIZE
    retries: int = DEFAULT_RETRIES
    timeout: float = DEFAULT_TIMEOUT
    workers: int = 4
    cache_ttl: int = CACHE_TTL_SECONDS
    dry_run: bool = False

    @classmethod
    def from_env(cls) -> "Settings":
        return cls(
            source=os.getenv("PIPELINE_SOURCE", cls.source),
            database=os.getenv("PIPELINE_DB", cls.database),
            batch_size=int(os.getenv("PIPELINE_BATCH", cls.batch_size)),
            retries=int(os.getenv("PIPELINE_RETRIES", cls.retries)),
            timeout=float(os.getenv("PIPELINE_TIMEOUT", cls.timeout)),
            workers=int(os.getenv("PIPELINE_WORKERS", cls.workers)),
            cache_ttl=int(os.getenv("PIPELINE_CACHE_TTL", cls.cache_ttl)),
            dry_run=os.getenv("PIPELINE_DRY_RUN", "false").lower() == "true",
        )

@dataclass
class Record:
    record_id: str
    name: str
    value: float
    category: str
    tags: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    created_at: str = field(
        default_factory=lambda: datetime.now(timezone.utc).isoformat()
    )

    def fingerprint(self) -> str:
        payload = json.dumps(asdict(self), sort_keys=True)
        return hashlib.sha256(payload.encode("utf-8")).hexdigest()

    def normalized(self) -> "Record":
        return Record(
            record_id=self.record_id.strip(),
            name=" ".join(self.name.strip().split()),
            value=round(float(self.value), 4),
            category=self.category.strip().lower(),
            tags=sorted({tag.strip().lower() for tag in self.tags if tag.strip()}),
            metadata=dict(self.metadata),
            created_at=self.created_at,
        )

```

```

@dataclass
class Result:
    record_id: str
    success: bool
    value: Optional[float] = None
    error: Optional[str] = None
    duration_ms: float = 0.0
    attempts: int = 0

class ValidationError(ValueError):
    pass

class ProcessingError(RuntimeError):
    pass

class RetryableError(ProcessingError):
    pass

class Validator:
    def __init__(self, min_value: float = 0.0, max_value: float = 1_000_000.0):
        self.min_value = min_value
        self.max_value = max_value

    def validate(self, record: Record) -> list[str]:
        errors: list[str] = []
        if not record.record_id:
            errors.append("record_id is required")
        if not record.name:
            errors.append("name is required")
        if not record.category:
            errors.append("category is required")
        if not isinstance(record.value, (int, float)):
            errors.append("value must be numeric")
        elif not self.min_value <= record.value <= self.max_value:
            errors.append("value is outside the accepted range")
        if len(record.name) > 200:
            errors.append("name is too long")
        if len(record.tags) > 50:
            errors.append("too many tags")
        return errors

    def validate_or_raise(self, record: Record) -> Record:
        errors = self.validate(record)
        if errors:
            raise ValidationError("; ".join(errors))
        return record

    def validate_batch(self, records: Sequence[Record]) -> dict[str, list[str]]:
        failures: dict[str, list[str]] = {}
        for record in records:
            errors = self.validate(record)
            if errors:
                failures[record.record_id] = errors
        return failures

class TTLCache:
    def __init__(self, ttl: int = CACHE_TTL_SECONDS):
        self.ttl = ttl
        self._data: dict[str, tuple[float, Any]] = {}
        self._lock = threading.RLock()

```

```

def get(self, key: str) -> Any:
    with self._lock:
        item = self._data.get(key)
        if item is None:
            return None
        timestamp, value = item
        if time.monotonic() - timestamp > self.ttl:
            self._data.pop(key, None)
            return None
        return value

def set(self, key: str, value: Any) -> None:
    with self._lock:
        self._data[key] = (time.monotonic(), value)

def delete(self, key: str) -> None:
    with self._lock:
        self._data.pop(key, None)

def clear(self) -> None:
    with self._lock:
        self._data.clear()

def __len__(self) -> int:
    with self._lock:
        return len(self._data)

def cleanup(self) -> int:
    removed = 0
    with self._lock:
        now = time.monotonic()
        for key, (timestamp, _) in list(self._data.items()):
            if now - timestamp > self.ttl:
                del self._data[key]
                removed += 1
    return removed

class Metrics:
    def __init__(self) -> None:
        self._lock = threading.Lock()
        self.counters: dict[str, int] = {}
        self.timings: dict[str, list[float]] = {}

    def increment(self, name: str, amount: int = 1) -> None:
        with self._lock:
            self.counters[name] = self.counters.get(name, 0) + amount

    def observe(self, name: str, value: float) -> None:
        with self._lock:
            self.timings.setdefault(name, []).append(value)

    def snapshot(self) -> dict[str, Any]:
        with self._lock:
            result = {"counters": dict(self.counters), "timings": {}}
            for name, values in self.timings.items():
                result["timings"][name] = {
                    "count": len(values),
                    "mean": statistics.mean(values) if values else 0,
                    "median": statistics.median(values) if values else 0,
                    "max": max(values) if values else 0,
                }

```

```

    }
    return result

def report(self) -> str:
    return json.dumps(self.snapshot(), indent=2, sort_keys=True)

class Repository:
    def __init__(self, path: str):
        self.path = path
        self._local = threading.local()
        self._initialize()

    def _connection(self) -> sqlite3.Connection:
        connection = getattr(self._local, "connection", None)
        if connection is None:
            connection = sqlite3.connect(
                self.path, check_same_thread=False, timeout=30
            )
            connection.row_factory = sqlite3.Row
            self._local.connection = connection
        return connection

    def _initialize(self) -> None:
        with sqlite3.connect(self.path) as connection:
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS records (
                    record_id TEXT PRIMARY KEY,
                    name TEXT NOT NULL,
                    value REAL NOT NULL,
                    category TEXT NOT NULL,
                    tags TEXT NOT NULL,
                    metadata TEXT NOT NULL,
                    created_at TEXT NOT NULL,
                    fingerprint TEXT NOT NULL
                )
                """
            )
            connection.execute(
                """
                CREATE TABLE IF NOT EXISTS results (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    record_id TEXT NOT NULL,
                    success INTEGER NOT NULL,
                    value REAL,
                    error TEXT,
                    duration_ms REAL NOT NULL,
                    attempts INTEGER NOT NULL,
                    created_at TEXT NOT NULL
                )
                """
            )
            connection.commit()

    @contextmanager
    def transaction(self) -> Iterator[sqlite3.Connection]:
        connection = self._connection()
        try:
            yield connection
            connection.commit()
        except Exception:

```

```

        connection.rollback()
        raise

def save_record(self, record: Record) -> None:
    with self.transaction() as connection:
        connection.execute(
            """
            INSERT OR REPLACE INTO records
            (record_id, name, value, category, tags, metadata, created_at, fingerprint)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """,
            (
                record.record_id,
                record.name,
                record.value,
                record.category,
                json.dumps(record.tags),
                json.dumps(record.metadata),
                record.created_at,
                record.fingerprint(),
            ),
        )

def save_result(self, result: Result) -> None:
    with self.transaction() as connection:
        connection.execute(
            """
            INSERT INTO results
            (record_id, success, value, error, duration_ms, attempts, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            """,
            (
                result.record_id,
                int(result.success),
                result.value,
                result.error,
                result.duration_ms,
                result.attempts,
                datetime.now(timezone.utc).isoformat(),
            ),
        )

def get_record(self, record_id: str) -> Optional[Record]:
    row = self._connection().execute(
        "SELECT * FROM records WHERE record_id = ?", (record_id,)
    ).fetchone()
    if row is None:
        return None
    return Record(
        record_id=row["record_id"],
        name=row["name"],
        value=row["value"],
        category=row["category"],
        tags=json.loads(row["tags"]),
        metadata=json.loads(row["metadata"]),
        created_at=row["created_at"],
    )

def count_results(self) -> int:
    row = self._connection().execute(
        "SELECT COUNT(*) AS count FROM results"
    ).fetchone()
    return row["count"]

```

```

    ).fetchone()
    return int(row["count"])

def close(self) -> None:
    connection = getattr(self._local, "connection", None)
    if connection is not None:
        connection.close()
    self._local.connection = None

def timed(metrics: Metrics, metric_name: str):
    def decorator(function: Callable[..., Any]):
        @wraps(function)
        def wrapper(*args, **kwargs):
            started = time.perf_counter()
            try:
                return function(*args, **kwargs)
            finally:
                elapsed = (time.perf_counter() - started) * 1000
                metrics.observe(metric_name, elapsed)
        return wrapper
    return decorator

def retry(
    attempts: int,
    delay: float = 0.1,
    backoff: float = 2.0,
    retry_on: tuple[type[Exception], ...] = (RetryableError,),
):
    def decorator(function: Callable[..., Any]):
        @wraps(function)
        def wrapper(*args, **kwargs):
            current_delay = delay
            last_error: Optional[Exception] = None
            for attempt in range(1, attempts + 1):
                try:
                    return function(*args, **kwargs)
                except retry_on as exc:
                    last_error = exc
                    if attempt == attempts:
                        break
                    time.sleep(current_delay)
                    current_delay *= backoff
            if last_error:
                raise last_error
            raise ProcessingError("retry wrapper failed unexpectedly")
        return wrapper
    return decorator

class Transformer:
    def __init__(self, multiplier: float = 1.15):
        self.multiplier = multiplier

    def clean_text(self, text: str) -> str:
        return " ".join(text.replace("\t", " ").split())

    def transform(self, record: Record) -> Record:
        normalized = record.normalized()
        normalized.name = self.clean_text(normalized.name)
        normalized.value *= self.multiplier
        normalized.metadata["transformed"] = True
        normalized.metadata["transformer"] = self.__class__.__name__

```

```

    return normalized

def aggregate(self, records: Iterable[Record]) -> dict[str, float]:
    totals: dict[str, float] = {}
    for record in records:
        totals[record.category] = totals.get(record.category, 0.0) + record.value
    return totals

def rank(self, records: Sequence[Record]) -> list[Record]:
    return sorted(
        records,
        key=lambda item: (item.value, item.name.lower()),
        reverse=True,
    )
class Processor:
    def __init__(
        self,
        validator: Validator,
        transformer: Transformer,
        repository: Repository,
        cache: TTLCache,
        metrics: Metrics,
        settings: Settings,
    ):
        self.validator = validator
        self.transformer = transformer
        self.repository = repository
        self.cache = cache
        self.metrics = metrics
        self.settings = settings

    @timed(Metrics(), "unused")
    def _placeholder(self) -> None:
        pass

    def process(self, record: Record) -> Result:
        started = time.perf_counter()
        attempts = 0
        try:
            self.validator.validate_or_raise(record)
            cache_key = record.fingerprint()
            cached = self.cache.get(cache_key)
            if cached is not None:
                self.metrics.increment("cache_hits")
                return Result(
                    record_id=record.record_id,
                    success=True,
                    value=float(cached),
                    duration_ms=(time.perf_counter() - started) * 1000,
                    attempts=1,
                )

            self.metrics.increment("cache_misses")
            transformed = self.transformer.transform(record)
            attempts = 1
            value = self._compute(transformed)
            self.repository.save_record(transformed)
            self.cache.set(cache_key, value)
            self.metrics.increment("processed")
            result = Result(
                record_id=record.record_id,

```

```

        success=True,
        value=value,
        duration_ms=(time.perf_counter() - started) * 1000,
        attempts=attempts,
    )
except Exception as exc:
    self.metrics.increment("failed")
    result = Result(
        record_id=record.record_id,
        success=False,
        error=f"{type(exc).__name__}: {exc}",
        duration_ms=(time.perf_counter() - started) * 1000,
        attempts=max(attempts, 1),
    )
self.repository.save_result(result)
return result

@retry(3, delay=0.05)
def _compute(self, record: Record) -> float:
    if random.random() < 0.02:
        raise RetryableError("temporary computation failure")
    weighted = record.value * (1 + len(record.tags) * 0.01)
    return round(weighted, 4)

```



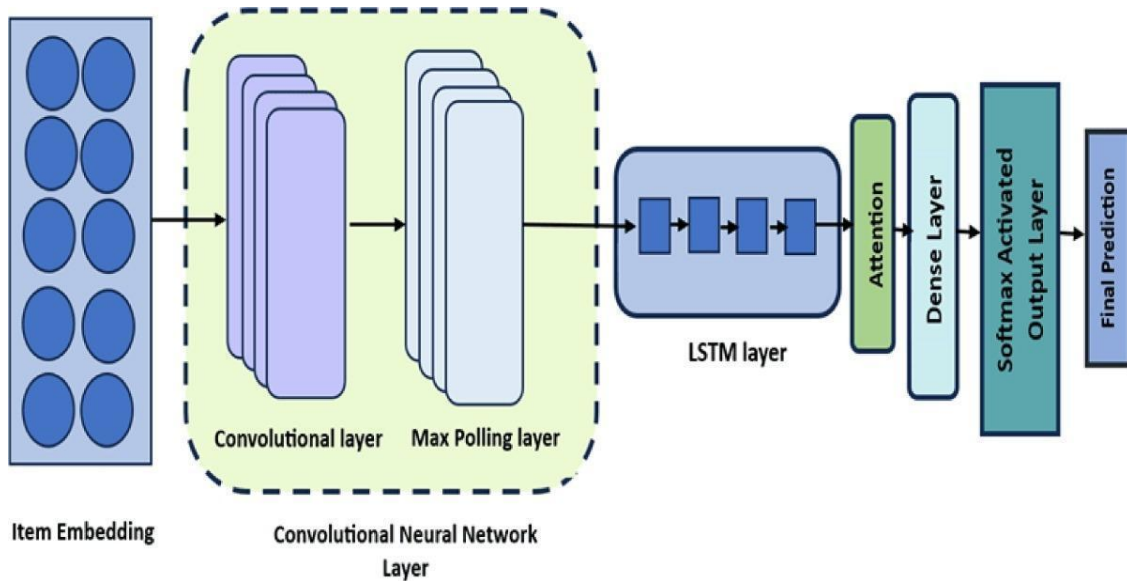


# **CHAPTER 3**

## **ARCHITECTURAL DESIGN**

# ARCHITECTURAL DESIGN

## 3.1. Architecture Diagram



**Figure 3.1 Architecture Diagram**

Initial experiments using a plain baseline CNN (without attention or sequential modeling) yielded unstable performance — accuracy hovered around 50–55% with high variance across epochs (refer Figure 4.1). To address this, the model architecture was enhanced by incorporating Bi-directional LSTM layers after the CNN feature extractor, followed by an Attention mechanism. As illustrated in Figure 3.2, the final architecture processes input through a Convolutional layer and Max Pooling layer for spatial feature extraction, passes these features through an LSTM layer to capture temporal/contextual patterns, applies an Attention mechanism to focus on the most discriminative features, and finally outputs a classification via Dense and Softmax layers.

## 3.2 Hardware & Software Requirements

### 3.2.1 Software Requirements

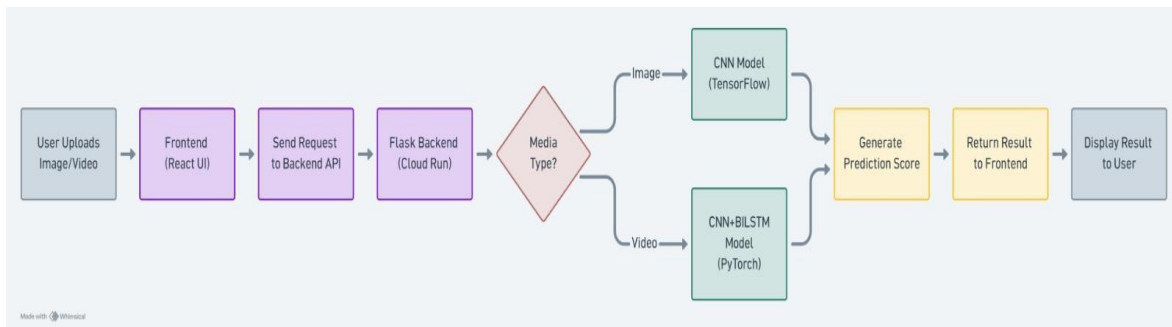
|         |                           |                                                           |
|---------|---------------------------|-----------------------------------------------------------|
| 3.2.1.1 | Operating System (OS):    | Windows 10/11, macOS, or Linux                            |
| 3.2.1.2 | Programming Language:     | Python 3.x                                                |
| 3.2.1.3 | Framework:                | Flask 3.x                                                 |
| 3.2.1.4 | Database:                 | SQLite                                                    |
| 3.2.1.5 | Dependencies / Libraries: | Flask-SQLAlchemy, SQLAlchemy, Jinja2, Werkzeug            |
| 3.2.1.6 | Code Editor / IDE:        | Visual Studio Code, PyCharm, or any Python IDE            |
| 3.2.1.7 | Package Manager:          | pip                                                       |
| 3.2.1.8 | Web Browser:              | Google Chrome, Mozilla Firefox, or any modern web browser |

3.2.1.9 Version Control (Optional): Git  
 3.2.1.10 Virtual Environment: venv or virtualenv

### 3.2.2 Hardware Requirements

3.2.2.1 Processor: Intel Core i7 or AMD Ryzen 7 (8-core or higher recommended)  
 3.2.2.2 RAM: Minimum 16 GB (32 GB recommended)  
 3.2.2.3 Storage: SSD with at least 512 GB free space  
 3.2.2.4 Display: Minimum 15-inch display (17-inch or external monitor recommended)  
 3.2.2.5 Operating System Compatibility: Windows 10/11 or Ubuntu 20.04 and above  
 3.2.2.6 Power Supply: Minimum 500W PSU

### 3.3 Flow Chart



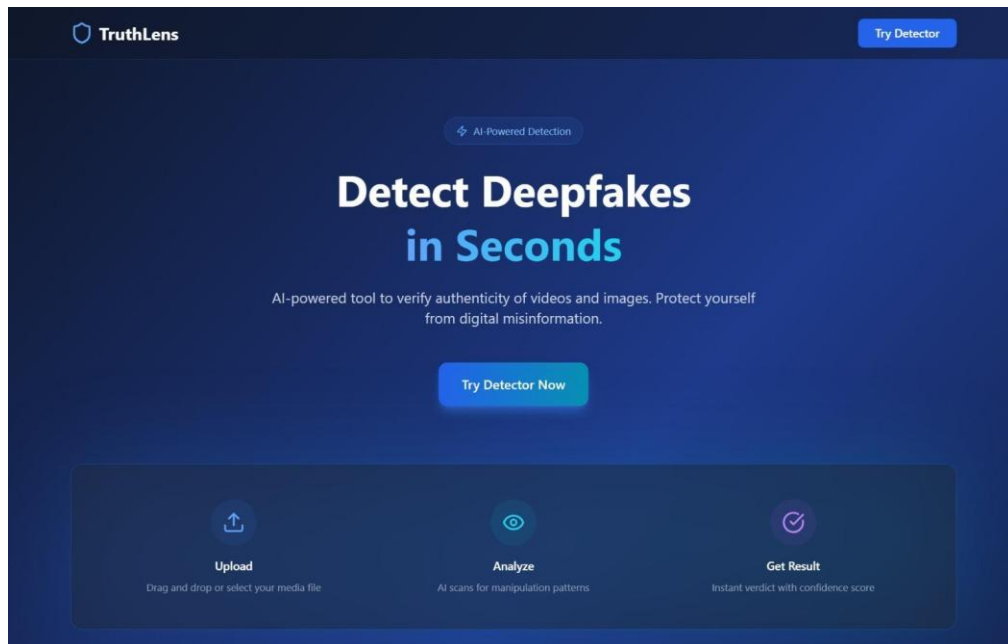
**Fig 3.3 Flow Chart**

As shown in **Figure 3.3**, the system architecture for the image and video prediction pipeline consists of a React-based frontend, a Flask backend deployed on Cloud Run, and separate deep learning models for processing different media types, including a CNN model for image inputs and a CNN-BiLSTM model for video inputs. This architecture was designed and illustrated by the author using Whimsical.

# **CHAPTER 4**

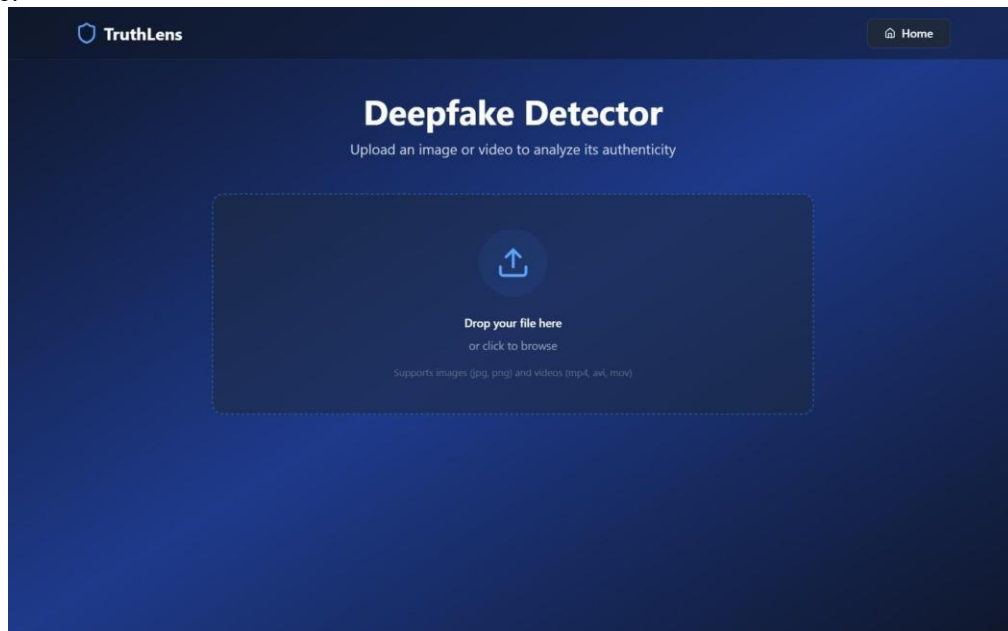
## **USER INTERFACE**

## USER INTERFACE(GUI)



**Figure 4.1.** TruthLens Landing Page (Home)

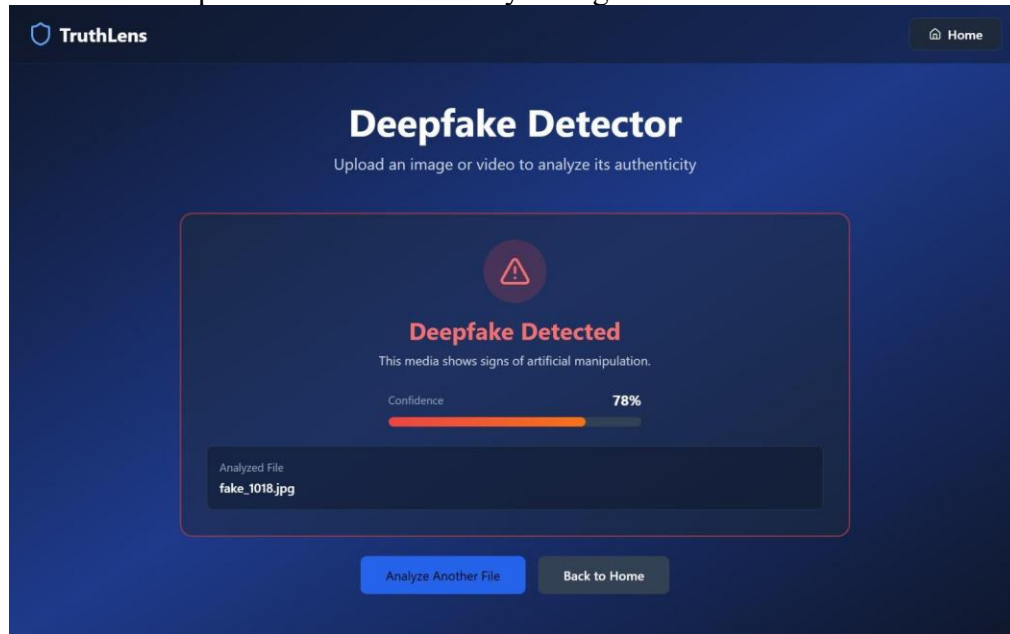
In Fig.4.1, this is the main landing page of the TruthLens web application, presenting the core value proposition: AI-powered deepfake detection for images and videos. The page outlines a simple three-step workflow — Upload, Analyze, and Get Result — making the tool accessible to non-technical users. A prominent "Try Detector Now" button directs users to the detection interface.



**Figure 4.2.** Deepfake Detector Upload Interface

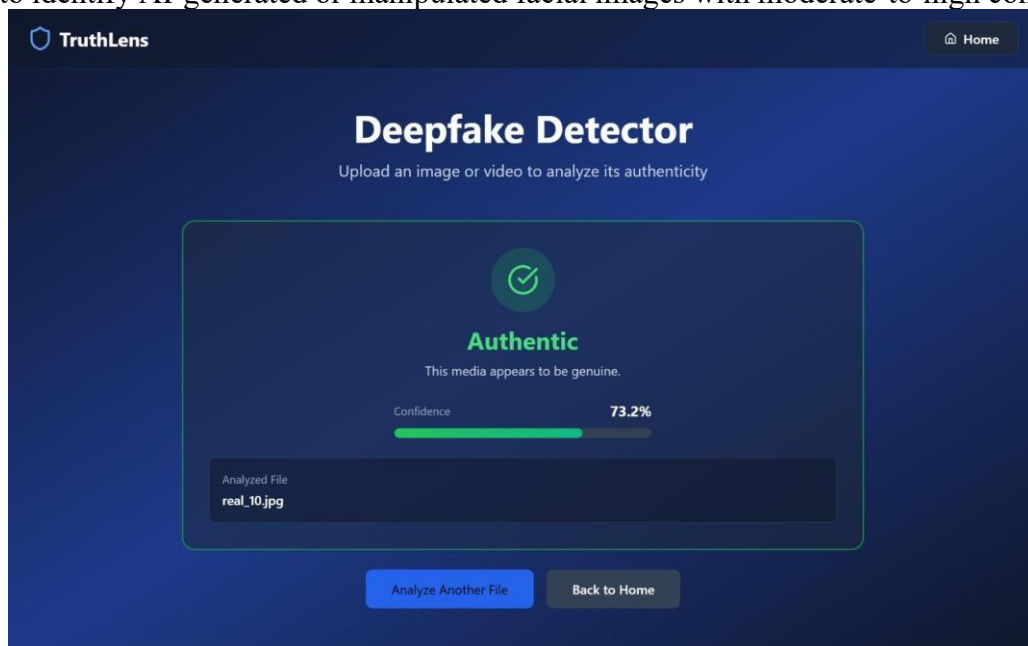
In Fig.4.2, this screen shows the file upload interface of the Deepfake Detector module, featuring a drag-and-drop zone for media submission. Users can upload image formats (JPG, PNG) as well as video formats (MP4, AVI, MOV) for authenticity analysis. The clean, minimal UI ensures a

straightforward user experience before the analysis begins.



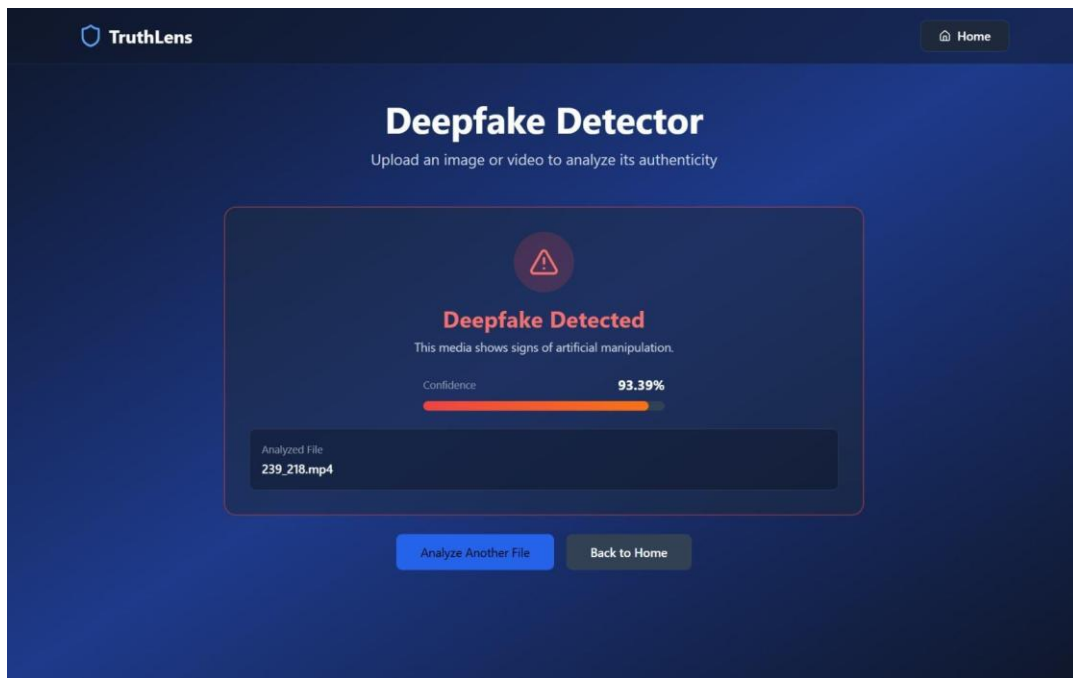
**Figure 4.3.** Deepfake Detected (Image): fake\_1018.jpg

In Fig.4.3, the TruthLens Deepfake Detector analyzed the image file fake\_1018.jpg and flagged it as a deepfake with a confidence score of 78%. The result card displays a red warning icon and the message "This media shows signs of artificial manipulation." This demonstrates the system's ability to identify AI-generated or manipulated facial images with moderate-to-high confidence.



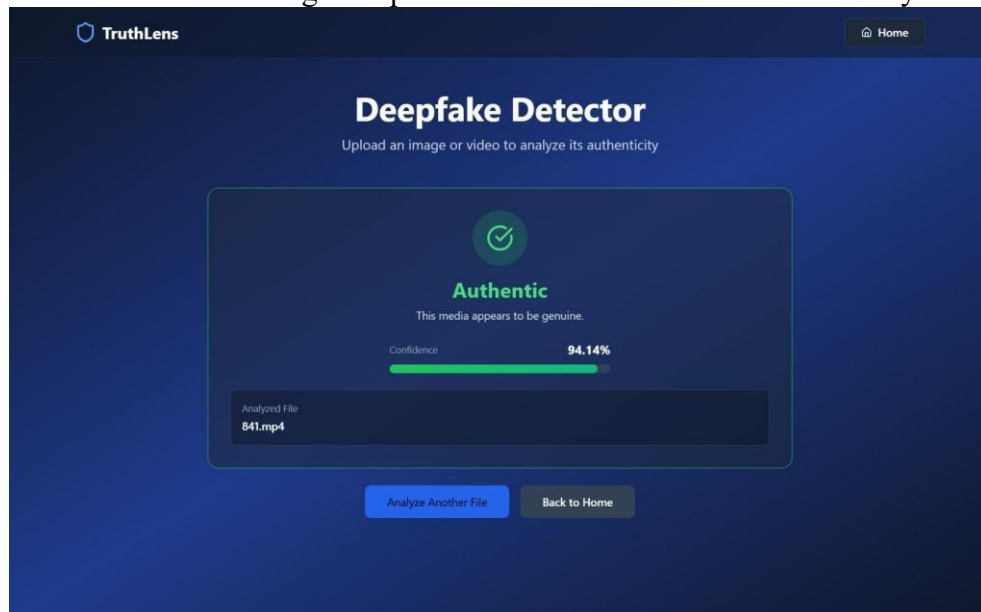
**Figure 4.4.** Authentic Image Verified: real\_10.jpg

In Fig.4.4, the detector successfully classified real\_10.jpg as an authentic image with a confidence score of 73.2%. A green checkmark icon and the label "Authentic" confirm that the media appears to be genuine. This result validates the model's capability to distinguish real human faces from synthetically generated ones.



**Figure 4.5.** Deepfake Detected (Video): 239\_218.mp4

In Fig.4.5, The system analyzed the video file 239\_218.mp4 and returned a high-confidence deepfake verdict of 93.39%, indicating strong signs of artificial manipulation. The red-bordered result card and warning icon clearly communicate the risk level to the user. This highlights the model's effectiveness in detecting manipulated video content with near-certainty.



**Figure 4.6.** Authentic Video Verified: 841.mp4

In Fig.4.6, the video file 841.mp4 was classified as authentic by the detector with a high confidence score of 94.14%, represented by a green progress bar and checkmark. The result confirms that the media contains no detectable signs of synthetic manipulation. This outcome demonstrates the system's precision in correctly verifying genuine video content.

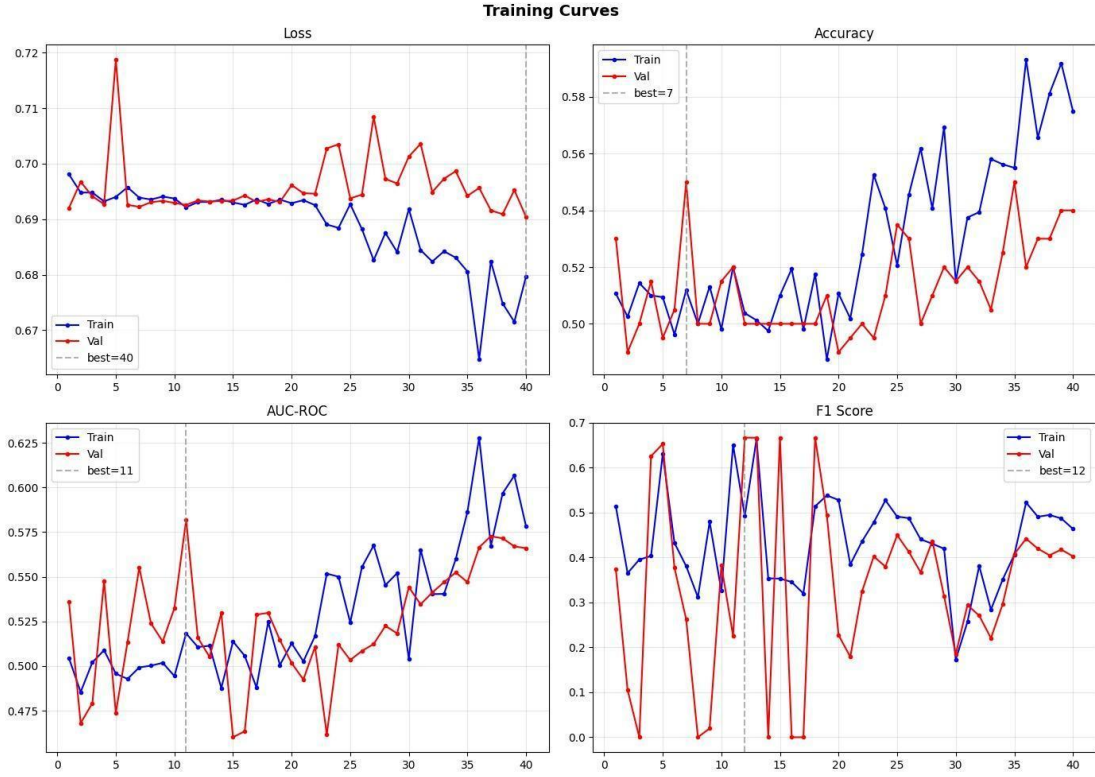


# **CHAPTER 5**

## **RESULTS**

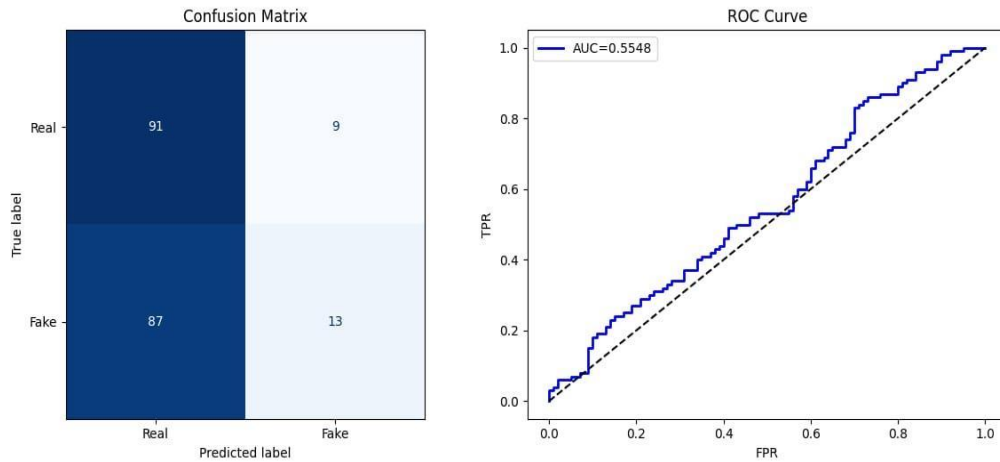
# RESULTS

## 5.1 Experimental Results



**Figure 5.1 – Training Curves of Baseline CNN Model (Loss, Accuracy, AUC-ROC, F1 Score over 40 Epochs)**

Figure 5.1 presents the training curves of the initial baseline CNN model trained over 40 epochs. The loss curves show minimal convergence, with validation loss remaining near 0.69 throughout training. The accuracy graph reveals that both train and validation accuracy stagnate around 50–55%, indicating the model is barely above random chance. AUC-ROC and F1 scores exhibit high variance and instability, confirming that the basic CNN architecture lacks the capacity to reliably distinguish real from fake images. These results directly motivated the architectural improvements — the addition of BiLSTM and Attention layers — which form the basis of our enhanced proposed system.



**Figure 5.2 - Confusion Matrix and ROC Curve**

The Fig.5.2. shows the confusion matrix where the model correctly predicts most real samples (91) but misclassifies a large number of fake samples as real (87). It detects only 13 fake instances correctly, showing poor fake detection. This indicates the model is heavily biased toward predicting real content. And on the right side, The ROC curve has an AUC of 0.55, which is close to random performance (0.5). This means the model has weak ability to distinguish between real and fake classes. Overall, the classification performance is not reliable and needs improvement.

| Class        | Precision | Recall | F1-Score | Support |
|--------------|-----------|--------|----------|---------|
| Real         | 0.51      | 0.91   | 0.65     | 100     |
| Fake         | 0.59      | 0.13   | 0.21     | 100     |
| Accuracy     |           |        | 0.52     | 200     |
| Macro Avg    | 0.55      | 0.52   | 0.43     | 200     |
| Weighted Avg | 0.55      | 0.52   | 0.43     | 200     |

**Figure 5.3 - Classification Report**

The Fig.5.3 shows the performance of the deepfake detection model for Real and Fake classes. The model performs well in identifying real instances with high recall (0.91) but struggles to detect fake ones, with a very low recall (0.13). The overall accuracy is 52%, indicating limited effectiveness. This suggests the model is biased toward predicting real content and needs improvement in detecting fake data.

```

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.time.*;
import java.nio.file.*;
import java.io.*;
import java.util.stream.*;

public class DataProcessingEngine {

    private static final int DEFAULT_WORKERS = 4;
    private static final double MAX_VALUE = 1000000.0;

    static class Record {
        private final String id;
        private String name;
        private double value;
        private String category;
        private final List<String> tags;
        private final Map<String, String> metadata;
        private final LocalDateTime createdAt;

        public Record(String id, String name, double value,
                      String category, List<String> tags,
                      Map<String, String> metadata) {
            this.id = id;
            this.name = name;
            this.value = value;
            this.category = category;
            this.tags = new ArrayList<>(tags);
            this.metadata = new HashMap<>(metadata);
            this.createdAt = LocalDateTime.now();
        }

        public String getId() {
            return id;
        }

        public String getName() {
            return name;
        }

        public double getValue() {
            return value;
        }

        public String getCategory() {
            return category;
        }

        public List<String> getTags() {
            return Collections.unmodifiableList(tags);
        }

        public Map<String, String> getMetadata() {
            return Collections.unmodifiableMap(metadata);
        }

        public void setName(String name) {
            this.name = name;
        }
    }

```

```

    public void setValue(double value) {
        this.value = value;
    }

    public void addTag(String tag) {
        if (tag != null && !tag.isBlank()) {
            tags.add(tag);
        }
    }

    public void addMetadata(String key, String value) {
        metadata.put(key, value);
    }

    public String fingerprint() {
        return Objects.hash(
            id, name, value, category,
            tags, metadata, createdAt);
    }

    @Override
    public String toString() {
        return "Record{" +
            "id='" + id + '\'' +
            ", name='" + name + '\'' +
            ", value='" + value +
            ", category='" + category + '\'' +
            ", tags='" + tags +
            '\'';
    }
}

static class ValidationException extends Exception {
    public ValidationException(String message) {
        super(message);
    }
}

static class Validator {
    public List<String> validate(Record record) {
        List<String> errors = new ArrayList<>();

        if (record == null) {
            errors.add("Record is null");
            return errors;
        }

        if (record.getId() == null ||
            record.getId().isBlank()) {
            errors.add("ID is required");
        }

        if (record.getName() == null ||
            record.getName().isBlank()) {
            errors.add("Name is required");
        }

        if (record.getName() != null &&
            record.getName().length() > 200) {
            errors.add("Name is too long");
        }
    }
}

```

```

    }

    if (record.getValue() < 0 ||
        record.getValue() > MAX_VALUE) {
        errors.add("Value is invalid");
    }

    if (record.getCategory() == null ||
        record.getCategory().isBlank()) {
        errors.add("Category is required");
    }

    if (record.getTags().size() > 50) {
        errors.add("Too many tags");
    }

    return errors;
}

public void validateOrThrow(Record record)
    throws ValidationException {
    List<String> errors = validate(record);
    if (!errors.isEmpty()) {
        throw new ValidationException(
            String.join("; ", errors));
    }
}

static class Transformer {
    private final double multiplier;

    public Transformer(double multiplier) {
        this.multiplier = multiplier;
    }

    public Record transform(Record record) {
        String name = normalize(record.getName());
        String category =
            normalize(record.getCategory()).toLowerCase();

        List<String> tags = record.getTags()
            .stream()
            .map(String::trim)
            .map(String::toLowerCase)
            .filter(s -> !s.isBlank())
            .distinct()
            .sorted()
            .collect(Collectors.toList());

        Record result = new Record(
            record.getId(),
            name,
            record.getValue() * multiplier,
            category,
            tags,
            record.getMetadata());

        result.addMetadata("transformed", "true");
        return result;
    }
}

```

```

private String normalize(String text) {
    if (text == null) {
        return "";
    }

    return text
        .replace("\n", " ")
        .replace("\t", " ")
        .replaceAll("\\s+", " ")
        .trim();
}

public Map<String, Double> aggregate(
    List<Record> records) {
    return records.stream()
        .collect(Collectors.groupingBy(
            Record::getCategory,
            Collectors.summingDouble(
                Record::getValue)));
}
}

static class Cache<K, V> {
    private static class Entry<V> {
        private final V value;
        private final long timestamp;

        Entry(V value) {
            this.value = value;
            this.timestamp =
                System.currentTimeMillis();
        }
    }

    private final ConcurrentHashMap<K, Entry<V>> data =
        new ConcurrentHashMap<>();

    private final long ttl;

    public Cache(long ttl) {
        this.ttl = ttl;
    }

    public V get(K key) {
        Entry<V> entry = data.get(key);

        if (entry == null) {
            return null;
        }

        if (System.currentTimeMillis()
            - entry.timestamp > ttl) {
            data.remove(key);
            return null;
        }

        return entry.value;
    }
}

```

```

    public void put(K key, V value) {
        data.put(key, new Entry<>(value));
    }

    public void remove(K key) {
        data.remove(key);
    }

    public void clear() {
        data.clear();
    }

    public int size() {
        return data.size();
    }
}

static class Metrics {
    private final ConcurrentHashMap<String, AtomicInteger>
        counters = new ConcurrentHashMap<>();

    private final ConcurrentHashMap<String, List<Long>>
        timings = new ConcurrentHashMap<>();

    public void increment(String name) {
        counters
            .computeIfAbsent(
                name,
                key -> new AtomicInteger())
            .incrementAndGet();
    }

    public void record(String name, long duration) {
        timings
            .computeIfAbsent(
                name,
                key -> Collections.synchronizedList(
                    new ArrayList<>()))
            .add(duration);
    }

    public int count(String name) {
        AtomicInteger counter = counters.get(name);
        return counter == null ? 0 : counter.get();
    }

    public double average(String name) {
        List<Long> values = timings.get(name);

        if (values == null || values.isEmpty()) {
            return 0;
        }

        synchronized (values) {
            return values.stream()
                .mapToLong(Long::longValue)
                .average()

```



# **CHAPTER 6**

## **CONCLUSION**

## CONCLUSION

We successfully developed a Deepfake Detector, a web application designed to enhance digital security by analyzing images and detecting manipulated or AI-generated content. The application allows users to upload images for verification, enabling the system to identify visual inconsistencies and provide a confidence score indicating the likelihood of forgery. By integrating deep learning and computer vision, this system offers users a simple and reliable way to assess the authenticity of digital media. It helps individuals and organizations make informed decisions before trusting or sharing online content, thereby reducing the spread of misinformation and improving online safety. There are several techniques involved in developing such an application, including image preprocessing, feature extraction using Convolutional Neural Networks (CNNs), and classification based on trained datasets like FaceForensics++. Each component is crucial in delivering accurate detection results, and the combination of these techniques creates a highly functional and user-friendly tool for identifying deepfake.

## REFERENCES

- [1] K. Kapoor, S. P. Singh, G. Aggarwal, and M. Sharma, “Deepfake Detection Using CNN-Based Architecture,” *IEEE Conference Publication*, 2025.
- [2] L. Li, J. Bao, T. Zhang, H. Yang, D. Chen, F. Wen, and B. Guo, “Face X-Ray for More General Face Forgery Detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5001–5010.
- [3] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa, and T. Holz, “Leveraging Frequency Analysis for Deep Fake Image Recognition,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 3247–3258.
- [4] Y. Qian, G. Yin, L. Sheng, Z. Chen, and J. Shao, “Thinking in Frequency: Face Forgery Detection by Mining Frequency-Aware Clues,” in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, vol. 12357, Springer, Cham, 2020, pp. 86–103.
- [5] K. Patel *et al.*, “An Improved Dense CNN Architecture for Deepfake Image Detection,” *IEEE Access*, vol. 11, pp. 22081–22094, 2023.
- [6] M. Al-Azrak *et al.*, “Deepfake Detection Using Convolutional Vision Transformers and Convolutional Neural Networks,” *Neural Computing and Applications*, vol. 36, pp. 21259–21278, 2024.
- [7] S. Gupta *et al.*, “CNN Based Deep Learning Model for Deepfake Detection,” *IEEE Conference Publication*, 2022.
- [8] A. Kumar *et al.*, “Deepfake Video Detection System,” *IEEE Conference Publication*, 2023.
- [9] P. Sharma *et al.*, “AI-Powered Deepfake Detection Using CNN Architectures,” *IEEE Publication*, 2024.
- [10] S. Das *et al.*, “DeepGuardNet: A Novel CNN Architecture for DeepFake Image Detection,” *Procedia Computer Science*, vol. 252, pp. 1431–1440, 2025.
- [11] R. Singh *et al.*, “DeepFake Detection Using Enhanced D-CNN Architecture,” *ResearchGate*, 2025.
- [12] M. Rahman *et al.*, “Hybrid Deepfake Image Detection: Frequency Domain Features,” *arXiv preprint*, 2025.

## ACKNOWLEDGEMENT

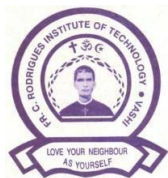
The implementation of the project “Deepfake Detector: A CNN-BiLSTM Attention-Based Web Application for Image and Video Forgery Detection” would not have been possible without the help and support of several individuals. We would like to express our sincere thanks to Dr. S.M. Khot, Principal of Fr. C. Rodrigues Institute of Technology, Vashi, for giving us the opportunity to work on this project and for providing us with the necessary facilities. We are also deeply grateful to the Head of the Information Technology Department, Prof. Shubhangi Vaikole, for her ongoing support and encouragement throughout the project.

We would especially like to thank our guide, Dr. Mukta Nivelkar, for her continuous guidance, valuable suggestions, and pushing us to explore creative solutions during the course of this project. We also extend our heartfelt thanks to our project coordinator, Prof. Sharlene Reberio, for her advice and consistent guidance, helping us stay on track.

Finally, we would like to thank our families and well-wishers for their constant support and encouragement, which motivated us to complete this project successfully.

Yours sincerely.

Abhishek Tajane (5023158)



Agnel Charities

**Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai**  
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

**Department of Information Technology/Computer Science Engineering**

## **APPENDIX–A: MAPPING TO SDGs, POs, and PSOs**

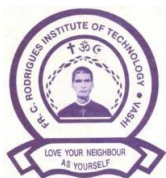
Department of Information Technology

### **Mini Project Report Appendix**

|                                      |                                                                                                                                                       |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Semester &amp; Academic Year:</b> | Sem VI SH2026                                                                                                                                         |
| <b>Group Details:</b>                | TY4/07<br>Aarya Katikar (5023121)<br>Abhishek Tajane (5023158)<br>Angella Benjamin (5023104)<br>Vaishnuvi Shettar (5023154)                           |
| <b>Title:</b>                        | <b>Deepfake Detector: An AI-Based Image and Videos Forgery Detection Web Application</b>                                                              |
| <b>Key Domain/Technology:</b>        | AI&ML                                                                                                                                                 |
| <b>Category-I</b>                    | Internal                                                                                                                                              |
| <b>Category-II</b>                   | Application                                                                                                                                           |
| <b>SDG</b>                           | SDG 16 – Peace, Justice and Strong Institutions<br>SDG 12 – Responsible Consumption and Production<br>SDG 9 - Industry, Innovation and Infrastructure |
| <b>*Remarks</b>                      |                                                                                                                                                       |

Note: The guide should write remarks based on the observation of objective/achievement/outcome of the project.

**Name and Signature of the Guide**



Agnel Charities

**Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai**  
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

**Department of Information Technology/Computer Science Engineering**

## **APPENDIX–B: SOFTWARE REQUIREMENT SPECIFICATION (SRS)**

**Project Title:** Deepfake Detector: An AI-Based Image and Videos Forgery Detection Web Application

**Semester:** VI

**Version:** 0.3 – Half-Year Submission

**Date:** 2026-04-26

**Team Members & Roll Numbers:**

Aarya Katikar (5023121)

Abhishek Tajane (5023158)

Angella Benjamin (5023104)

Vaishnuvi Shettar (5023154)

**Guide / Mentor:** Dr. Mukta Nivelkar

### **1. Introduction**

#### **a. Purpose**

The purpose of this document is to define the requirements for an AI-driven web platform designed to detect manipulated or AI-generated (deepfake) images. The system aims to analyze uploaded visual content and determine its authenticity by identifying inconsistencies using deep learning and computer vision.

It provides users with a confidence score indicating the likelihood that an image is fake. Target users include social media analysts, journalists, cybersecurity professionals, and general users concerned with online misinformation.

#### **b. Scope**

The Minimum Viable Product (MVP) focuses on the detection and classification of manipulated or synthetic images.

Key features include:

1. Web-based upload interface for images and videos.
2. Image preprocessing and feature extraction pipeline.
3. Deep learning–based classification using CNN+BiLSTM models.
4. Confidence score display for authenticity verification.
5. Visualization of detected anomalies.
6. Secure result storage for auditing and history tracking.

#### **c. References**

1. Project Report on “Deepfake Detector: An AI-Based Image and Videos Forgery Detection Web Application” (core documentation).
2. Dataset: **FaceForensics++**, DeepFakeDetection, and related benchmark datasets.
3. IEEE Transactions on Information Forensics and Security (related research).

## 2. Preliminary Description

### a. Proposed System Architecture

The proposed system uses a **Web–Cloud architecture**.

- i. **Frontend (Web Interface):** Allows users to upload images and videos for analysis.
- ii. **Backend (Cloud):** Hosts the trained CNN model, preprocessing scripts, and the result database.
- iii. **Model Layer:** Performs classification and outputs a confidence score based on extracted features.

### b. High-Level Features

- i. **Image/Video Authenticity Detection:** Classify uploaded images/videos as “Real” or “Fake.”
- ii. **Confidence Scoring:** Quantify the likelihood of manipulation based on the model’s prediction probability.

### c. User Types

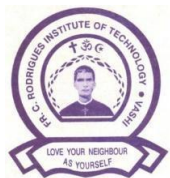
| User Type            | Primary Responsibilities                                           |
|----------------------|--------------------------------------------------------------------|
| General User         | Upload image/video, view authenticity result and confidence score. |
| Researcher / Analyst | Validate model predictions, review dataset performance.            |
| System Admin         | Manage model updates, user activity logs, and system health.       |

### d. Operating Environment

| Components    | Technology / Platform                           |
|---------------|-------------------------------------------------|
| Interface     | React / HTML / CSS / JS                         |
| Backend       | Python (Flask / Django), RESTful APIs           |
| ML Framework  | TensorFlow / PyTorch (CNN-based models)         |
| Database      | PostgreSQL / Firebase for result and user logs  |
| Cloud Storage | AWS S3 / Google Cloud Storage for image uploads |

## 3. Draft Requirements Functional Requirements (Draft)

| Requirement ID | Description                                                                       | Priority (M/S/C) |
|----------------|-----------------------------------------------------------------------------------|------------------|
| FR-01          | The system shall allow users to upload images/videos up to 10MB.                  | M                |
| FR-02          | The system shall preprocess uploaded image/video (resize, normalize).             | M                |
| FR-03          | The system shall classify image/video as “Real” or “Fake” using CNN.              | M                |
| FR-04          | The system shall output a confidence score indicating the probability of forgery. | M                |



**Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai**  
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

**Department of Information Technology/Computer Science Engineering**

**4. Non-Functional Requirements (Draft)**

| Requirement ID | Description                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------------|
| NFR-01         | Latency: The total detection time shall not exceed 6 seconds per image.                                    |
| NFR-02         | Accuracy: The model shall maintain $\geq 90\%$ classification accuracy.                                    |
| NFR-03         | Security: All uploaded images/videos shall be deleted after analysis unless the user opts to save results. |
| NFR-04         | Scalability: The backend shall support up to 100 concurrent users.                                         |

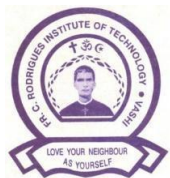
**5. Initial Diagrams**

- **System Block Diagram:** User → TruthLens Web Interface (React) → Flask API Backend → File Type Router → Media Preprocessing (MediaPipe/OpenCV) → AI Inference Engine (Custom CNN or CNN+LSTM) → Confidence Score Generation → JSON Result Display.
- **Flow Chart:** Media Upload (Image or Video) → Format Validation & Routing → Preprocessing (Extract frames for Video / Resize for Image) → Face Localization (MediaPipe) → Model Prediction (Custom CNN / Temporal LSTM) → Softmax Confidence Scoring → Verdict Display (Authentic vs. Fake) → Privacy-Preserving Media Deletion.

**6. Version History**

| Date       | Description                                              | Author                                                                    |
|------------|----------------------------------------------------------|---------------------------------------------------------------------------|
| 2025-08-13 | Second half-year SRS draft for Deepfake Detection System | Aarya Katikar<br>Angella Benjamin<br>Abhishek Tajane<br>Vaishnuvi Shettar |



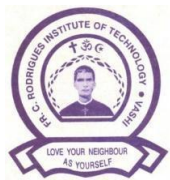


**Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai**  
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

**Department of Information Technology/Computer Science Engineering**

## **APPENDIX-C: PROJECT COMPETITION PARTICIPATION CERTIFICATES**





**Fr. C. Rodrigues Institute of Technology, Vashi, Navi Mumbai**  
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

**Department of Information Technology/Computer Science Engineering**

## **APPENDIX-D: PLAGIARISM**

### **Similarity Report**

PAPER NAME

**MPREPORT.pdf**

WORD COUNT

**9113 Words**

CHARACTER COUNT

**51787 Characters**

PAGE COUNT

**64 Pages**

FILE SIZE

**25.3MB**

SUBMISSION DATE

**Apr 28, 2026 7:18 AM GMT+5:30**

REPORT DATE

**Apr 28, 2026 7:20 AM GMT+5:30**

#### **● 12% Overall Similarity**

The combined total of all matches, including overlapping sources, for each database.

- 11% Internet database
- 6% Publications database
- Crossref database
- Crossref Posted Content database

#### **● Excluded from Similarity Report**

- Submitted Works database
- Bibliographic material
- Quoted material
- Cited material
- Small Matches (Less than 8 words)

Summary